# Fast Flow Engine

## FFENGINE

## Agent Architecture Document

FAD — AI Agent Skill Geliştirme Referans Dokümanı

TAD v6.0 Tabanlı

Versiyon
FAD v6.0
Baz Doküman
FFEngine TAD v6.0
Dönem
2026-Q1
Doküman Tipi
FAD — Fonksiyonel & Agent Architecture Document
Hedef
AI Agent skill dosyaları için kaynak doküman
Platform
ffengine Community (Query-Centric) + Enterprise (Queue-Aware)

# İçindekiler

# 00 — Doküman Amacı ve Agent Kullanım Kılavuzu

Bu doküman, FFEngine TAD v6.0 içeriğini temel alarak AI agent'ların ffengine için skill dosyaları üretebilmesi, kod oluşturabilmesi ve mimari kararlar verebilmesi için gerekli tüm bağlamı sağlar. Her bölüm, agent'ın ihtiyaç duyduğu kararları net kurallar ve bağlam tabloları ile tanımlar.

> **Agent Kullanım Prensibi:** Bu doküman okunduğunda agent, (1) Community ile Enterprise arasındaki tüm özellik sınırlarını, (2) her bileşenin hangi versiyona ait olduğunu, (3) YAML konfigürasyon kurallarını ve (4) Delivery Guarantee koşullarını eksiksiz olarak bilir. Herhangi bir skill dosyası üretmeden önce bu dokümanın ilgili bölümü referans alınmalıdır.

## Doküman Kapsamı

| Bölüm | İçerik | Agent Kullanım Amacı |
|---|---|---|
| 00 | Doküman amacı ve kullanım rehberi | Başlangıç bağlamı |
| 01 | Ürün kimliği ve mimari karar çerçevesi | Versiyon sınırı kararları |
| 02 | Community mimarisi (Query-Centric) | Community skill üretimi |
| 03 | Enterprise mimarisi (Queue-Aware) | Enterprise skill üretimi |
| 04 | Engine-Swap katman haritası | Ortak vs ayrışan bileşen kararları |
| 05 | Veri akış protokolü ve worker mimarisi | Pipeline konfigürasyon üretimi |
| 06 | Dialect ve TypeMapper katmanı | DB-specific kod üretimi |
| 07 | YAML konfigürasyon tam referansı | Config dosyası üretimi |
| 08 | Mapping dosya yapısı | Mapping YAML üretimi |
| 09 | Airflow entegrasyonu ve DAG pattern | DAG kodu üretimi |
| 10 | Hata yönetimi ve delivery guarantee matrisi | Error handling skill üretimi |
| 11 | Teknoloji stack ve bağımlılıklar | Dependency ve setup skill üretimi |
| 12 | Gereksinim ve kapsam sınır tablosu | Neyin içeride neyin dışarıda olduğu |

# 01 — Ürün Kimliği ve Mimari Karar Çerçevesi

ffengine iki ayrı mimari yaklaşımı tek bir ürün çatısı altında sunar. Agent'ın her özellik veya kod parçası için önce versiyonu belirlemesi, ardından o versiyonun kurallarını uygulaması gerekir.

## Birincil Karar Kuralı — Versiyon Ayrımı

**Kural #1 — Agent İçin:** Bir özellik, bileşen veya yapılandırma üretmeden önce şu soruyu sor: 'Bu Community mu Enterprise mı?' Cevap net değilse aşağıdaki Özellik Sınır Tablosu'na başvur. Yanlış versiyona atama yapılmış bir skill dosyası yanlış beklenti yaratır.

## Mimari Kimlik Tablosu

| Boyut | ffengine community | ffengine enterprise |
|---|---|---|
| Mimari Yaklaşım | Query-Centric Batch ETL | Queue-Aware / Capability-First Data Integration Engine |
| Çekirdek Dil | Python 3.12 / Standard DBAPI 2.0 | C / C++ + Python extension veya bağımsız binary |
| Aktarım Motoru | Python Engine: fetchmany → executemany döngüsü | C Engine: Ingress/Egress Queue + native bulk API + Zero-Copy |
| Hedef Kitle | Veri mühendisleri, orta-büyük ölçekli ETL ihtiyaçları | Finansal sistemler, kritik yük, yüksek throughput gereksinimleri |
| Lisanslama | Open Source (PyPI / Docker Hub) | Ticari — RSA şifreli lisans modülü, Binary Wheels / Private Registry |
| Lansman Dönemi | Q1 2026 | Q2 2026 |
| Delivery Guarantee | Best-effort; load_method ile idempotency sağlanır | At-Least-Once (varsayılan); Exactly-Once (koşullu — Guarantee Matrix) |

## Özellik Sınır Tablosu — Hızlı Referans

**Agent Kullanımı:** Aşağıdaki tablo, bir özelliğin Community'de mi, Enterprise'da mı, yoksa her ikisinde de mi bulunduğunu tek bakışta gösterir. Skill dosyası üretirken bu tabloyu kontrol et.

| Özellik / Bileşen | Community | Enterprise | Notlar |
|---|---|---|---|
| fetchmany + executemany | ✓ | ✓ (fallback) | Enterprise C Engine yoksa community path çalışır |
| Server-side cursor (backpressure) | ✓ | ✓ | Her iki versiyonda ortak |
| YAML konfigürasyon / ETL Studio | ✓ | ✓ | Ortak katman — değişmez |
| Apache Airflow FFEngineOperator | ✓ | ✓ | Ortak — automatic fallback |

| Özellik / Bileşen | Community | Enterprise | Notlar |
|---|---|---|---|
| 3 Fazlı DAG Pattern | ✓ | ✓ | Ortak |
| Partitioning (auto/percentile/hash_mod/explicit/distinct) | ✓ | ✓ | Ortak |
| Auto-DDL Generator / TypeMapper | ✓ | ✓ | Ortak dialect katmanı |
| validate_config() | ✓ | ✓ | Ortak |
| Email Notification / Dry-Run | ✓ | ✓ | Ortak |
| COPY FROM STDIN (Binary) — PG hedef | ✗ | ✓ | Enterprise Only |
| COPY TO STDOUT (Binary) — PG kaynak | ✗ | ✓ | Enterprise Only |
| BCP / fast_executemany — MSSQL | ✗ | ✓ | Enterprise Only |
| OCI_BATCH_ERRORS — Oracle | ✗ | ✓ | Enterprise Only |
| Zero-Copy Streaming (C buffer) | ✗ | ✓ | Enterprise Only |
| Ingress Queue / Egress Queue | ✗ | ✓ | Enterprise Only |
| FFEnvelope / Ack-Nack | ✗ | ✓ | Enterprise Only |
| Checkpoint & Resume | ✗ | ✓ | Enterprise Only |
| Dead Letter Queue (DLQ) | ✗ | ✓ | Enterprise Only |
| Adaptive Micro-Batch (row+byte+time) | ✗ | ✓ | Enterprise Only |
| Multi-Lane Pipeline | ✗ | ✓ | Enterprise Only |
| Ordered Delivery / Exactly-Once (koşullu) | ✗ | ✓ | Enterprise Only — Guarantee Matrix koşulları gerekir |
| N Reader / M Writer (asimetrik worker) | ✗ | ✓ | Enterprise Only |
| SIMD optimizasyonu | ✗ | ✓ | Enterprise Only |
| Kafka / RabbitMQ entegrasyonu | ✗ | ✗ (gelecek faz) | Mevcut fazda kapsam dışı |
| CDC / WAL / Redo Log | ✗ | ✗ | Kalıcı kapsam dışı |
| Veri maskeleme (KVKK/GDPR) | ✗ | ✗ | Kalıcı kapsam dışı |
| Veri kalitesi kuralları (regex, null) | ✗ | ✗ | Kalıcı kapsam dışı |

# 02 — Community Mimarisi (Query-Centric Batch ETL)

ffengine community, query-centric bir yaklaşımla çalışır: standart DBAPI döngüsü (fetchmany ile sorgula, executemany ile yaz) tüm veri aktarımının temelini oluşturur. Native bulk API yoktur; delivery guarantee best-effort'tur.

## Community Veri Akış Modeli

| Adım | Bileşen | Teknoloji | Açıklama |
|---|---|---|---|
| 1 — Extract | SourceReader | Server-side cursor (fetchmany) | Kaynak DB'de cursor açılır; veri fetchmany(10000) ile chunk'lar halinde çekilir. Tablo asla tamamen RAM'e alınmaz. |
| 2 — Buffer | Buffer Engine (Streamer) | Python generator (yield) | Chunk'lar yield ile producer-consumer pattern üzerinden geçirilir. Basit backpressure: hedef yazma yavaşlarsa üretim throttle edilir. |
| 3 — Transform | Transformer / TypeMapper | TABLE_RULES, TypeMapper | On-the-fly tip dönüşümü. strip, parse_dates, to_numeric, empty_to_null, add_etl_date kuralları. |
| 4 — Load | TargetWriter | executemany (standart DBAPI) | Hedef DB'ye standart batch yazma. DB'ye özel hızlı API yok; serileştirme overhead mevcuttur. |
| 5 — Orchestrate | FFEngineOperator | Apache Airflow 3.1.6 | Görev tetikleme, XCom metrikleri (rows, duration, throughput), 3 fazlı DAG pattern. |

## Community Hata Yönetimi

| Durum | Davranış | Not |
|---|---|---|
| Chunk yazma hatası | Chunk bazlı transaction rollback; hata loglanır, aktarım durur | DLQ yok; retry mekanizması Airflow task retry'ına bırakılmıştır |
| Hatalı satır | Hata Airflow log'a yazılır; işlem durur | Partial success handling yok — Enterprise Only |
| Bağlantı kopması | Airflow task başarısız olarak işaretlenir | Checkpoint & Resume yok — baştan başlamak gerekir |
| Konfigürasyon hatası | validate_config() çalışma başında yakalar | Çalışma zamanı hatası önlenir |
| OOM riski | fetchmany ile chunk bazlı okuma sayesinde düşük | 500MB altı RAM hedefi |

## Community Sınırları — Agent'ın Bilmesi Gereken

**Agent Uyarısı:** Aşağıdaki yetenekler Community için üretilen skill dosyalarına DAHIL EDİLMEMELİDİR. Bu özellikleri içeren bir Community skill yanlış beklenti yaratır.

- COPY / BCP / OCI_BATCH — native bulk yazma
- Checkpoint & Resume (PK/Offset bazlı)
- Dead Letter Queue (DLQ)
- Ingress/Egress Queue, FFEnvelope, Ack/Nack
- Adaptive Micro-Batch (row+byte+time limiti)
- Multi-Lane Pipeline, Ordered Delivery
- Exactly-Once delivery semantiği
- extraction_method: copy_binary (bu Community'de çalışmaz)

# 03 — Enterprise Mimarisi (Queue-Aware / Capability-First)

ffengine enterprise, queue-aware ve capability-first bir mimariye sahiptir. Klasik batch ETL'den farklı olarak her iki uçta da queue semantiği (backpressure, ack/nack, checkpoint, DLQ, delivery policy) uygulanır. C Engine ve native bulk API'ler maksimum throughput sağlar.

## Enterprise Veri Akış Modeli (Queue-Centric)

| Adım | Bileşen | Teknoloji | Açıklama |
|---|---|---|---|
| 1 — Produce | Source Producer | COPY TO STDOUT (PG) / native API | Kaynak DB'den küçük binary paketler üretilir ve Ingress Queue'ya bırakılır. Her paket FFEnvelope içinde sarmalanır. |
| 2 — Ingress | Ingress Queue | Thread-safe queue, FFEnvelope | Backpressure kontrolü ile üretim hızı yönetilir. Checkpoint kaydı tutulur; paket ack/nack ile işaretlenir. |
| 3 — Transform | Transformer / TypeMapper | TABLE_RULES, TypeMapper (ortak) | On-the-fly dönüşüm — Community ile aynı katman. |
| 4 — Egress | Egress Queue | Delivery Policy, CheckpointStore | Paket teslimat politikasına göre (ordered/at-least-once/exactly-once koşullu) Target Consumer'a iletilir. |
| 5 — Consume | Target Consumer | COPY FROM STDIN / BCP / OCI_BATCH | Native bulk API ile kontrollü mikro-batch commit. Adaptive Micro-Batch politikası aktif. |
| 6 — Orchestrate | FFEngineOperator | Apache Airflow 3.1.6 (ortak) | Aynı operatör; C Engine otomatik tespit edilir (automatic fallback). |

## Enterprise Çekirdek Nesneleri

| Nesne | Modül | Sorumluluk | Anahtar Özellikler |
|---|---|---|---|
| FFEnvelope | Queue Runtime | Her veri paketi için standart sarmalayıcı | header (metadata, partition_key, sequence) + payload (data batch) |
| IngressQueue | Queue Runtime | Kaynak→Transform köprüsü, backpressure | Thread-safe, maxsize=pipe_queue_max, otomatik throttle |
| EgressQueue | Queue Runtime | Transform→Hedef köprüsü, delivery policy | Ordered/at-least-once/exactly-once (koşullu) semantik |
| CheckpointStore | Queue Runtime | Offset ve ack takibi, resume yeteneği | Source offset + target write ack; PK/Offset bazlı ilerleme kaydı |
| DeliveryManager | Queue Runtime | Paket teslimat stratejisi yönetimi | Delivery policy uygular, retry sayısını takip eder |
| BackpressureContr | Queue Runtime | Üretim/tüketim hız | Queue doluluk oranına |

| Nesne | Modül | Sorumluluk | Anahtar Özellikler |
|---|---|---|---|
| oller | | dengeleme | göre producer throttle |
| SourceAdapter | Ingress Side | Kaynak sistemden veri alma adaptörü | COPY TO STDOUT (PG), SQL Polling, Oracle AQ (gelecek faz) |
| TargetConsumer | Egress Side | Hedefe native bulk API ile yazma | COPY FROM STDIN, BCP, OCI_BATCH; adaptif mikro-batch commit |

## Adaptive Micro-Batch Politikası

Sabit satır sayısı yerine çoklu limit kombinasyonu kullanılır. Hangisi önce dolarsa paket kapatılır; hedef sistem yavaşlarsa batch küçültülür.

| Limit Tipi | Varsayılan | YAML Parametresi | Açıklama |
|---|---|---|---|
| Satır Sayısı | 500 satır | batch_policy.max_rows | Paket başına maksimum satır sayısı |
| Byte Limiti | 2 MB | batch_policy.max_bytes | Paket başına maksimum veri boyutu |
| Transaction Süresi | 2 saniye | batch_policy.max_time | Transaction açık kalma süresi üst sınırı |
| Hedef Yanıt Adaptasyonu | Dinamik | — (otomatik) | Hedef yavaşlarsa lane başına batch boyutu küçültülür |
| Lock Süresi Adaptasyonu | Dinamik | — (otomatik) | Lock süresi artarsa concurrency otomatik düşürülür |

## Multi-Lane Pipeline — YAML Yapısı

```
# Enterprise Only — Multi-Lane Pipeline Tanımı
lanes:
  - lane_id: "orders_lane_1"
    source_adapter: "oracle_aq"      # veya: sql_polling, pg_copy_stdout
    target_writer: "pg_micro_batch"  # veya: mssql_bcp, oracle_oci_batch
    checkpoint: true
    pool_limit: 3
    retry_policy: "exponential_backoff"
    ordering_key: "order_id"
    batch_policy:
      max_rows: 500
      max_bytes: "2MB"
      max_time: "2s"
```

# 04 — Engine-Swap Katman Haritası

ffengine, değiştirilebilir Engine mimarisi üzerine kuruludur. Ortak katmanlar her iki versiyonda aynı çalışır; yalnızca veri taşıma motoru (Engine) değişir. Agent, skill dosyası üretirken hangi katmanın ortak, hangisinin versiyona özgü olduğunu net bilmelidir.

| Katman | Ortak mu? | Community İmplementasyonu | Enterprise İmplementasyonu |
|---|---|---|---|
| ETL Studio UI | ✓ Ortak | Admin > ETL Studio Airflow plugin | ← Aynı |
| Dialect / Agnostic Layer | ✓ Ortak | PostgresDialect, OracleDialect, MSSQLDialect | ← Aynı |
| TypeMapper | ✓ Ortak | Merkezi tip eşleme (YAML bazlı) | ← Aynı |
| DB Bağlantı (DBSession) | ✓ Ortak | psycopg3, python-oracledb, pyodbc | Native OCI, libpq, SQL Server Native Client |
| YAML Konfigürasyon | ✓ Ortak | Aynı şema, aynı parametreler | ← Aynı (extraction_method: copy_binary Enterprise'da aktif) |
| FFEngineOperator | ✓ Ortak | Python Engine tetiklenir | C Engine tespit edilir (automatic fallback) |
| Auto-DAG Generator | ✓ Ortak | YAML → Airflow DAG | ← Aynı |
| 3 Fazlı DAG Pattern | ✓ Ortak | plan → prepare → run | ← Aynı |
| validate_config() | ✓ Ortak | Çalışma öncesi YAML doğrulama | ← Aynı |
| Metadata Inspector | ✓ Ortak | SchemaReader, DDLGenerator | ← Aynı |
| Partitioning | ✓ Ortak | auto/percentile/ hash_mod/explicit/ distinct | ← Aynı (worker sayısı Enterprise'da anlamlı) |
| Buffer Engine (Streamer) | ✗ Ayrışır | Python generator (yield) — basit backpressure | Ingress Queue + Egress Queue — tam queue semantiği |
| Extraction Metodu | ✗ Ayrışır | cursor: fetchmany (her zaman) | copy_binary: COPY TO STDOUT (PG) / cursor (cross-DB) |
| Hedef Yazma | ✗ Ayrışır | executemany (standart DBAPI) | COPY FROM STDIN / BCP / OCI_BATCH (native bulk API) |
| Delivery Guarantee | ✗ Ayrışır | Best-effort; chunk rollback | At-Least-Once (default); Exactly-Once (koşullu) |
| Checkpoint / DLQ | ✗ Ayrışır | Yok | CheckpointStore + DLQ Policy |
| Adaptive Micro-Batch | ✗ Ayrışır | Yok — sabit batch_size | Evet — |

| Katman | Ortak mu? | Community İmplementasyonu | Enterprise İmplementasyonu |
|---|---|---|---|
| | | | row+byte+time+feedback |
| C Engine | ✗ Ayrışır | Yok | Zero-Copy C buffer, SIMD, native drivers |

**Automatic Fallback Kuralı:** FFEngineOperator, engine='auto' (varsayılan) olduğunda ortamda C Engine kütüphanesinin yüklü olup olmadığını kontrol eder. Yüklüyse Enterprise C Engine tetiklenir; yüklü değilse Community Python Engine devreye girer. Bu sayede aynı YAML konfigürasyonu ile her iki versiyonda da çalışılabilir.

# 05 — Veri Akış Protokolü ve Worker Mimarisi

## Streaming & Bulk Mekanizmaları — Versiyon Ayrımı

| Mekanizma | Versiyon | Yön | Açıklama |
|---|---|---|---|
| Server-Side Cursor (fetchmany) | Community + Enterprise | Okuma | fetchmany(10000) — sabit RAM, OOM engelleme |
| executemany (standart DBAPI) | Community | Yazma | Tüm DB'lerde çalışır; serileştirme overhead mevcuttur |
| COPY TO STDOUT (FORMAT BINARY) | Enterprise Only | Okuma | PG kaynak; Python nesnesine dönüşüm yok; ~2-3x hızlı |
| COPY FROM STDIN (FORMAT BINARY) | Enterprise Only | Yazma | PG hedef; %30-40 düşük CPU; en hızlı PG yükleme |
| BCP / fast_executemany | Enterprise Only | Yazma | MSSQL; standart executemany'ye göre ~10x hız |
| OCI_BATCH_ERRORS | Enterprise Only | Yazma | Oracle native; hatalı satırı izole ederek devam |
| Zero-Copy C Streaming | Enterprise Only | Okuma+Yazma | C buffer'ları üzerinden; Python nesne overhead sıfır |
| Backpressure (generator) | Community | Kontrol | yield + buffer.qsize >= pipe_queue_max → time.sleep |
| Backpressure (queue semantics) | Enterprise | Kontrol | Queue maxsize + producer block + adaptive throttle |

## Asimetrik N Reader / M Writer Mimarisi — Enterprise

Kaynak ve hedef sistemlerin I/O kapasiteleri genellikle farklıdır. reader_workers ve writer_workers parametreleri bağımsız ayarlanarak her iki tarafın kapasitesi optimize edilir.

| Parametre | Tip | Varsayılan | Versiyon | Açıklama |
|---|---|---|---|---|
| reader_workers | int | 3 | Enterprise | Kaynaktan paralel okuyan worker sayısı. Enterprise: COPY TO STDOUT; Community: fetchmany |
| writer_workers | int | 5 | Enterprise | Hedefe paralel yazan worker sayısı. Enterprise: native bulk API; Community: executemany |
| pipe_queue_max | int | 8 | Community + Enterprise | Reader-writer arası kuyruk derinliği. Backpressure kontrol mekanizması. |
| extraction_method | enum | auto | Community + Enterprise | copy_binary (Enterprise Only, PG) / cursor (her iki versiyon) / auto |

| Parametre | Tip | Varsayılan | Versiyon | Açıklama |
|---|---|---|---|---|
| batch_size | int | 10000 | Community + Enterprise | fetchmany chunk boyutu (cursor modda) |

| Senaryo | Reader | Writer | Gerekçe |
|---|---|---|---|
| Hızlı kaynak, yavaş hedef (PG → MSSQL) | 2 | 5 | Kaynakta az bağlantı; hedefte çoklu yazıcı ile baskı dağıtılır |
| Yavaş kaynak, hızlı hedef (Oracle → PG) | 5 | 2 | Paralel okuma ile kaynak darboğazı giderilir |
| PG → PG (Enterprise, aynı hız) | 3 | 3 | Simetrik: COPY TO STDOUT → Queue → COPY FROM STDIN |
| PG → PG (Community) | — | — | reader/writer_workers Community'de etkisiz; fetchmany + executemany |
| Büyük tablo, DWH yükleme | 4 | 8 | Yazma tarafında yoğun commit baskısını dağıtmak için |

## Partitioning Stratejileri

| Mod | Sınıf | Kullanım Senaryosu | Her İki Versiyon? |
|---|---|---|---|
| auto | AutoNumericPartition | Kolon tipine göre otomatik alt strateji seçimi | ✓ |
| percentile | PercentileRangePartition | Büyük, dağınık veri setleri — eşit bölümleme | ✓ |
| hash_mod | HashModuloPartition | Uniform dağılımlı kolonlar — deterministic | ✓ |
| explicit | ExplicitRangePartition | Tarih/değer bazlı, kullanıcı tanımlı sınırlar | ✓ |
| distinct | DistinctValuePartition | Kategori/enum kolonları — benzersiz değerlere göre | ✓ |
| full_scan | _FullScanStrategy | Küçük tablolar / WHERE ile sınırlı — tek parça | ✓ |

# 06 — Dialect Soyutlama ve TypeMapper Katmanı

Dialect katmanı ortak bir katmandır — Community ve Enterprise arasında değişmez. Her veritabanı için BaseDialect'ten türetilen bir sınıf yazılır. Agent, yeni DB desteği eklerken bu yapıyı temel almalıdır.

## BaseDialect Arayüzü — Implement Edilmesi Gereken Metodlar

| Metod | Zorunlu mu? | Açıklama | Agent Notu |
|---|---|---|---|
| get_table_schema(schema, table) | ✓ | Kaynaktan metadata (kolon adı, tip, nullable) çekme | ALL_TAB_COLUMNS / information_schema / sys.columns |
| generate_bulk_insert_query(schema, table, cols) | ✓ | Hedef DB'nin en hızlı INSERT sentaksını üret | Community: standart INSERT \| Enterprise: COPY/BCP/OCI |
| get_data_type_map() | ✓ | Kaynak → hedef tip eşlemesi sözlüğü | TypeMapper tarafından kullanılır |
| generate_ddl(table_meta) | ✓ | Kaynak metadata'dan hedef CREATE TABLE DDL'i üret | Auto-DDL Generator tarafından kullanılır |
| get_pagination_query(sql, offset, limit) | ✓ | DB-specific pagination sentaksı | OFFSET/FETCH, ROWNUM, ROW_NUMBER |
| generate_bulk_extract_query(inner_sql) | Enterprise Only | COPY TO STDOUT sentaksı (yalnızca PostgresDialect) | f'COPY ({inner_sql}) TO STDOUT (FORMAT BINARY)' |

## Dialect Sınıfı — Metadata API'ları

| Veritabanı | Dialect Sınıfı | Metadata View / API | Notlar |
|---|---|---|---|
| PostgreSQL | PostgresDialect | information_schema.columns, pg_catalog | COPY TO STDOUT Enterprise'da ek metod olarak eklenir |
| Oracle | OracleDialect | ALL_TAB_COLUMNS, ALL_TABLES | Oracle 38 basamak precision — TypeMapper'da dikkat |
| MSSQL | MSSQLDialect | sys.columns, sys.objects, INFORMATION_SCHEMA | pyodbc / UnixODBC; fast_executemany=True Enterprise Only |

## TypeMapper — Veri Tipi Eşleme Tablosu

| Oracle Tipi | MSSQL Tipi | PostgreSQL Tipi | Kritik Not |
|---|---|---|---|
| NUMBER(p,s) | DECIMAL(p,s) | NUMERIC(p,s) | Finansal: p ve s değerleri korunmalı; kayıp kabul edilemez |
| NUMBER(16,0) | BIGINT | BIGINT | Scale=0 → integer tipi; otomatik tespit |
| VARCHAR2(n) | NVARCHAR(n) | VARCHAR(n) | Unicode desteği Oracle'da implicit; MSSQL'de N prefix |
| DATE | DATETIME2 | TIMESTAMP | Sürücü seviyesinde datetime objesi; string manipulation yok |
| CLOB | NVARCHAR(MAX) | TEXT | LOB chunking stratejisi gerekli — belirli boyut limitine kadar |
| BLOB | VARBINARY(MAX) | BYTEA | LOB chunking stratejisi gerekli |
| TIMESTAMP(n) | DATETIME2(n) | TIMESTAMP(n) | Mikrosaniye hassasiyeti; n parametresi korunmalı |
| CHAR(n) | CHAR(n) | CHAR(n) | Trailing space davranışı farklı — karşılaştırma hatalarına yol açabilir |

**Agent Notu — LOB Yönetimi:** CLOB/BLOB tipleri streaming sırasında belleği patlatabilir. Bu kolonlar ayrı bir chunking stratejisiyle taşınmalı ya da belirli bir boyut limitine kadar desteklenmelidir. Skill dosyasında LOB tipi tespit edildiğinde uyarı üretilmeli ve chunking stratejisi önerilmelidir.

## Transformer TABLE_RULES

| Kural | Uygulama Zamanı | Açıklama | Her İki Versiyon? |
|---|---|---|---|
| strip | on-the-fly | String kolonlardaki baş/son boşlukları temizler | ✓ |
| parse_dates | on-the-fly | Tarih string'lerini datetime nesnesine dönüştürür | ✓ |
| to_numeric | on-the-fly | String sayısal değerleri numeric'e cast eder | ✓ |

| Kural | Uygulama Zamanı | Açıklama | Her İki Versiyon? |
|---|---|---|---|
| empty_to_null | on-the-fly | Boş string ('') değerleri NULL'a çevirir | ✓ |
| add_etl_date | on-the-fly | Her satıra otomatik ETL timestamp kolonu ekler | ✓ |

# 07 — YAML Konfigürasyon Tam Referansı

Tüm ETL görevleri YAML dosyaları üzerinden tanımlanır. Agent, config dosyası üretirken bu bölümü referans almalıdır. Parametre versiyonu, zorunluluğu ve varsayılan değeri tabloda belirtilmiştir.

## Tam Konfigürasyon Şablonu

```
# ffengine ETL Konfigürasyonu — Tam Şablon
source_db_var: ""               # Airflow Connection ID (kaynak DB)
target_db_var: ""               # Airflow Connection ID (hedef DB)

etl_tasks:
  - task_group_id: "src_schema_src_table_to_tgt_schema_tgt_table_v1"

    # ── Kaynak Tanımı ──────────────────────────────────────
    source_schema: "source_schema"
    source_table:  "source_table_name"
    source_type:   table           # table | view | sql | csv | script
    column_mapping_mode: "source"  # source | mapping_file
    sql_file: ""                   # harici .sql yolu (source_type: sql ile)

    # ── Hedef Tanımı ───────────────────────────────────────
    target_schema: "target_schema"
    target_table:  "target_table_name"
    load_method: "create_if_not_exists_or_truncate"

    # ── Filtreleme & Bağlamalar ────────────────────────────
    where: >
      "col" > :binding_param
    order_by: ""
    bindings:
      binding_param:
        from: target              # source | target | literal | airflow_var
        sql: "SELECT MAX(col) FROM schema.table"
        default: "1900-01-01"

    # ── Streaming (Community + Enterprise) ─────────────────
    passthrough_full:   true      # tüm kolonları aktar
    passthrough_format: "binary"  # binary | text | csv
    pipe_queue_max:     8         # backpressure buffer derinliği
    batch_size:         10000     # fetchmany chunk boyutu

    # ── Worker Mimarisi (Enterprise) ───────────────────────
    extraction_method: "auto"     # copy_binary(E) | cursor(C+E) | auto
    reader_workers:    3          # Enterprise: COPY TO STDOUT paralel worker
    writer_workers:    5          # Enterprise: native bulk API paralel writer

    # ── Partitioning ───────────────────────────────────────
    partitioning:
      enabled: true
      column:  "id"
      mode:    "percentile"       # auto|percentile|hash_mod|explicit|distinct
      parts:   3

    # ── DAG Metadata ───────────────────────────────────────
    tags: ["domain", "level1", "src_to_stg"]
```

## Parametre Referans Tablosu

| Parametre | Tip | Zorunlu | Varsayılan | Versiyon | Açıklama |
|---|---|---|---|---|---|
| task_group_id | string | ✓ | — | C+E | Benzersiz görev kimliği; dosya adlandırma ile uyumlu olmalı |
| source_schema | string | ✓ | — | C+E | Kaynak DB şema adı |
| source_table | string | ✓ | — | C+E | Kaynak tablo adı |
| source_type | enum | — | table | C+E | table / view / sql / csv / script |
| column_mapping_mode | enum | — | source | C+E | source (kolon aynen) / mapping_file (YAML mapping) |
| sql_file | string | — | — | C+E | Harici .sql yolu; source_type: sql ile kullanılır |
| tags | list | — | [] | C+E | DAG tag'leri; klasör yapısından otomatik türetilir |
| target_schema | string | ✓ | — | C+E | Hedef DB şema adı |
| target_table | string | ✓ | — | C+E | Hedef tablo adı |
| load_method | enum | — | truncate | C+E | Aşağıda detaylı tablo |
| where | string | — | — | C+E | SQL WHERE koşulu; binding parametreleri ile kullanılır |
| bindings | map | — | — | C+E | source / target / literal / airflow_var |
| passthrough_full | bool | — | true | C+E | Tüm kolonları aktar |
| passthrough_format | enum | — | binary | C+E | binary / text / csv |
| pipe_queue_max | int | — | 8 | C+E | Backpressure buffer derinliği |
| batch_size | int | — | 10000 | C+E | fetchmany chunk boyutu (satır) |
| extraction_method | enum | — | auto | C+E | copy_binary → Enterprise Only / cursor → C+E / auto |
| reader_workers | int | — | 3 | Enterprise | Paralel COPY TO STDOUT worker sayısı |
| writer_workers | int | — | 5 | Enterprise | Paralel native bulk API writer sayısı |
| partitioning.mode | enum | — | auto | C+E | auto / percentile / hash_mod / explicit / distinct |
| partitioning.parts | int | — | 2 | C+E | Partition parça sayısı |

## load_method Seçenekleri

| load_method | Davranış | Kullanım Senaryosu |
|---|---|---|
| create_if_not_exists_or_truncate | Tablo yoksa CREATE, varsa TRUNCATE + INSERT | Varsayılan staging modu |
| append | Mevcut veriye INSERT | Incremental / delta yükleme |
| replace | DROP + CREATE + INSERT | Tam tablo yenileme |
| upsert | PK bazlı INSERT + UPDATE | Delta; duplicate önleme; idempotency için önerilir |
| delete_from_table | WHERE bazlı DELETE + INSERT | Belirli kayıtları silip yeniden yaz |
| drop_if_exists_and_create | DROP IF EXISTS + CREATE + INSERT | Her çalışmada sıfırdan oluştur |
| script | Hedef DB'de .sql script çalıştır | source_type: script ile birlikte kullanılır |

# 08 — Mapping Dosya Yapısı

column_mapping_mode: mapping_file seçildiğinde her görev için bağımsız bir YAML mapping dosyası kullanılır. mapping_generator.py CLI aracı ile otomatik üretilir; ETL Studio UI üzerinden de oluşturulabilir.

## Dizin Hiyerarşisi

```
mappings/
  └── {source_schema}/
        └── {source_table}_to_{target_schema}_{target_table}_v{N}.yaml

# Örnek:
mappings/
  └── ocn_iss/
        ├── crd_account_balance_used_hist_to_dwh_stg_v1.yaml
        └── txn_issuer_security_to_dwh_stg_txn_issuer_security_v1.yaml
```

## Mapping Dosyası — Parametre Şeması

| Alan | Tip | Zorunlu | Açıklama |
|---|---|---|---|
| version | string | ✓ | Şema versiyonu (ör: '1.0') |
| mapping_type | enum | ✓ | column (tek desteklenen değer) |
| generated_at | ISO 8601 | — | Otomatik üretim timestamp'i |
| source.schema | string | ✓ | Kaynak şema adı |
| source.table | string | ✓ | Kaynak tablo adı |
| source.sql | string | — | Custom SQL (opsiyonel; tablo adı yerine) |
| columns[].source | string | ✓ | Kaynak kolon adı |
| columns[].target | string | ✓ | Hedef kolon adı |
| columns[].type | string | ✓ | Hedef veri tipi — TypeMapper ile uyumlu |

## Mapping Dosyası — Örnek (Tablo Bazlı)

```yaml
version: '1.0'
mapping_type: column
generated_at: '2025-10-07T10:14:49Z'
source:
  schema: ocn_iss
  table:  crd_account_balance_used_hist
columns:
  - source: guid
    target: guid
    type:   NUMERIC(16, 0)
  - source: last_updated
    target: last_updated
    type:   NUMERIC(16, 0)
  - source: account_no
    target: account_no
    type:   VARCHAR(10)
```

## Mapping Dosyası — Örnek (Custom SQL)

```
version: '1.0'
mapping_type: column
generated_at: '2025-10-08T12:10:10Z'
source:
  schema: ocn_iss
  table:  txn_issuer_security
  sql: |
    SELECT t.txn_guid, t.cavv_ind, t.cavv_data, t.security_level_indicator
    FROM "ocn_iss"."txn_issuer_security" t
    WHERE EXISTS (
      SELECT 1 FROM ocn_iss.txn_issuer_onl tio
      WHERE t.txn_guid = tio.guid
    )
columns:
  - source: txn_guid
    target: txn_guid
    type:   NUMERIC(16, 0)
  - source: cavv_data
    target: cavv_data
    type:   VARCHAR(48)
```

# 09 — Airflow Entegrasyonu ve DAG Pattern

## FFEngineOperator — Tam Şablon

```
from airflow.models import BaseOperator
from airflow.utils.decorators import apply_defaults

class FFEngineOperator(BaseOperator):
    """
    ffengine Custom Airflow Operator.
    engine='auto': C Engine yüklüyse Enterprise, değilse Community.
    """
    @apply_defaults
    def __init__(
        self,
        config_path: str,       # YAML dosya yolu
        source_conn_id: str,    # Airflow Connection ID (kaynak)
        target_conn_id: str,    # Airflow Connection ID (hedef)
        task_group_id: str,     # Çalıştırılacak task_group_id
        engine: str = "auto",   # "community" | "enterprise" | "auto"
        **kwargs
    ):
        super().__init__(**kwargs)
        self.config_path    = config_path
        self.source_conn_id = source_conn_id
        self.target_conn_id = target_conn_id
        self.task_group_id  = task_group_id
        self.engine         = engine

    def execute(self, context):
        engine = _detect_engine(self.engine)  # automatic fallback
        result = engine.run(self.config_path, self.task_group_id)
        # XCom metrikleri
        self.xcom_push(context, key='rows_transferred', value=result.rows)
        self.xcom_push(context, key='duration_seconds', value=result.duration)
        self.xcom_push(context, key='rows_per_second',  value=result.throughput)
        return result
```

## 3 Fazlı DAG Execution Pattern — Tam Şablon

```
from airflow.decorators import dag, task, task_group
from airflow.operators.trigger_dagrun import TriggerDagRunOperator

for etl_task in config["etl_tasks"]:
    with TaskGroup(group_id=etl_task["task_group_id"]) as tg:

        @task
        def plan_partitions(task_cfg):
            # Faz 1: Binding'leri çöz + partition spec listesi üret
            resolved = resolve_where_with_bindings(task_cfg)
            # from: source | target | literal | airflow_var
            specs = partitioner.generate(resolved)
            return specs  # XCom → [{"where": "id BETWEEN 0 AND 50000"}, ...]

        @task
        def prepare_target(task_cfg):
            # Faz 2: Hedef tabloyu hazırla (load_method'a göre DDL)
            target_writer.prepare(task_cfg["load_method"])

        @task
        def run_partition(partition_spec, task_cfg):
            # Faz 3: Her partition bağımsız worker'da çalışır
            ETLManager.run_etl_task(task_cfg, partition_spec)
```

```
        specs = plan_partitions(etl_task)
        prep  = prepare_target(etl_task)
        specs >> prep >> run_partition.expand(partition_spec=specs)
```

## Binding Çözümleme — from Kaynakları

| from Değeri | Kaynak | Örnek Kullanım |
|---|---|---|
| source | Kaynak veritabanı | Kaynak DB'den MAX tarih gibi değerler |
| target | Hedef veritabanı | Hedef DB'deki son yükleme tarihi (DWH watermark) |
| literal | Sabit değer | Sabit tarih veya string parametre |
| airflow_var | Airflow Variable | Runtime'da Airflow Variables tablosundan çekilir |

## TriggerDagRunOperator — Kademeli Tetikleme

```
# Level1 (src→stg) tamamlandığında Level2 (stg→final) tetiklenir
trigger_level2 = TriggerDagRunOperator(
    task_id="trigger_level2_stg_to_whk",
    trigger_dag_id="whk_level2_stg_to_whk",
    wait_for_completion=True,  # Level2 bitene kadar bekle
    poke_interval=30,
)
all_level1_tasks >> trigger_level2
```

## ETL Studio UI Özellikleri

| Özellik | Açıklama | Her İki Versiyon? |
|---|---|---|
| Şema/Tablo Keşfi | Kaynak DB'den şema+tablo listesi; 50 tablo limiti; typeahead arama | ✓ |
| DAG Oluşturma | UI üzerinden YAML + DAG dosyası görsel üretimi; SQL ayrı .sql dosyasına | ✓ |
| DAG Güncelleme | Mevcut DAG'ler sonradan güncellenebilir | ✓ |
| Tag Yönetimi | Klasör yapısından otomatik tag; UI'den manuel ekleme | ✓ |
| Timeline Görünümü | Çalışan DAG'ler timeline olarak görselleştirilir | ✓ |
| Enterprise Performans Metrikleri | rows/sec, queue depth, lag anlık görselleştirme | Enterprise |

## Proje Dizin Yapısı

```
projects/{proje_adı}/
  └── {domain}/
        ├── level1/
        │   ├── src_to_stg/
        │   │   ├── config.yaml    # Ana ETL konfigürasyonu
        │   │   ├── mappings/      # YAML mapping dosyaları
        │   │   └── sql/           # Harici SQL dosyaları
```

```
        │   └── src_to_ods/
        │       └── config.yaml
        └── level2/
            └── stg_to_final/
                ├── config.yaml
                └── sql/           # DELETE + INSERT pattern

# Klasör → DAG tag otomatik türetme:
# whk/level1/src_to_stg → tags: ["whk", "level1", "src_to_stg"]
```

# 10 — Hata Yönetimi ve Delivery Guarantee Matrisi

## Hata Yönetimi — Versiyon Karşılaştırması

| Özellik | Community | Enterprise |
|---|---|---|
| Hatalı chunk | Transaction rollback; Airflow task başarısız | Ack/Nack contract; retry policy; DLQ'ya yönlendirme |
| Bağlantı kopması | Task başarısız; Airflow task retry | Checkpoint'ten resume; source offset korunur |
| Kısmi başarı | Yok — tümü veya hiç | Partial success; hatalı paket DLQ'ya; diğerleri devam |
| Yeniden çalışma | Baştan başlar | Checkpoint'ten kaldığı yerden devam |
| Poison message | Task başarısız | Max retry sonrası DLQ'ya izole edilir |
| Loglama | Airflow DAG log; JSON formatında | Airflow DAG log + queue depth/lag metrikleri |
| Seviye bazlı log | Level 1=INFO, 2=WARNING, 3=ERROR | Level 1=INFO, 2=WARNING, 3=ERROR |
| Email notification | on_failure_callback (Türkçe HTML) | on_failure_callback (Türkçe HTML) |
| Dry-Run modu | DDL üretimi + bağlantı testi | DDL üretimi + bağlantı testi + queue check |

## Delivery Guarantee Matrisi — Enterprise Only

**Temel Prensip:** Exactly-once FFEngine tarafından garantilenmez; altyapı koşullarının bütünü sağlandığında ortaya çıkan bir sonuçtur. FFEngine at-least-once'ı her zaman garantiler. Community'de delivery guarantee best-effort'tur — load_method ile idempotency sağlanır.

| Kombinasyon | Aktif Bileşenler | Semantik | Öneri |
|---|---|---|---|
| Queue + Idempotent Sink + Ordering Key | Ingress/Egress Queue + CheckpointStore + ordering_key + hedef PK/UNIQUE | Exactly-Once (Koşullu) | Kritik finansal veriler için — en güvenli kombinasyon |
| Queue + Idempotent Sink (ordering yok) | Ingress/Egress Queue + CheckpointStore + hedef PK/UNIQUE | At-Least-Once + No-Duplicate (sırasız) | Sıra önemsizse yeterli; duplicate oluşmaz |
| Native Queue (AQ/Service Broker) + idempotent write | Native queue + Ack/Nack + hedef idempotency | Exactly-Once (Koşullu) | Hedef idempotency zorunlu |
| Native Queue + hedef idempotency YOK | Native queue + Ack/Nack (hedef PK yok) | At-Least-Once | Retry'da duplicate oluşabilir |
| Cross-DB Bulk Writer + checkpoint | Egress Queue + COPY/BCP/OCI + | At-Least-Once | load_method: upsert ile |

| Kombinasyon | Aktif Bileşenler | Semantik | Öneri |
|---|---|---|---|
| | checkpoint | | duplicate giderilebilir |
| Cross-DB Bulk Writer + load_method: upsert | Egress Queue + native bulk API + UPSERT | Effectively Exactly-Once | Teknik at-least-once; upsert ile pratik exactly-once davranışı |
| Cross-DB Bulk Writer + load_method: truncate+insert | Egress Queue + native bulk API + TRUNCATE+INSERT | Exactly-Once (Idempotent Yeniden Yüklenebilir) | Staging/DWH için önerilir; partial failure'da tablo boş kalır |
| Checkpoint YOK | Egress Queue + native bulk API (checkpoint devre dışı) | Best-Effort / At-Most-Once | Kritik iş yüklerinde kullanılmamalı |
| Community — load_method: upsert | executemany + UPSERT hedef mantığı | Effectively Exactly-Once (uygulama seviyesinde) | Duplicate oluşmaz; sıra garantisi yok |
| Community — load_method: truncate+insert | TRUNCATE + executemany INSERT | Idempotent Yeniden Yüklenebilir | Her çalışmada tablo sıfırlanır; staging için uygun |

**Agent Uygulama Rehberi:** Kritik finansal veriler için: Queue + Idempotent Sink + Ordering Key + Checkpoint kombinasyonu. Staging/DWH için: truncate+insert. Incremental delta için: upsert. Community'de Checkpoint/DLQ içeren bir skill üretilmemelidir.

# 11 — Teknoloji Stack ve Bağımlılıklar

| Python 3.12 | C / C++ | Airflow 3.1.6 | psycopg3 |
|---|---|---|---|
| Community dili | Enterprise çekirdeği | Orkestrasyon | PG sürücüsü |

| Kategori | Teknoloji | Versiyon | Kapsam | Kurulum Notu |
|---|---|---|---|---|
| Orkestrasyon | Apache Airflow | 3.1.6 | C+E | Docker: apache/airflow:3.1.6-python3.12 |
| Python | Python | 3.12 | C+E | Minimum 3.10+ |
| PostgreSQL sürücüsü | psycopg3 | 3.x | C+E (farklı özellik seti) | pip install psycopg[binary]; Community: standart DBAPI; Enterprise: binary COPY |
| MSSQL sürücüsü | pyodbc + msodbcsql18 | — | Community | UnixODBC; apt install unixodbc-dev |
| Oracle sürücüsü | python-oracledb | — | Community | Thin mode — Oracle Client kurulumu gerekmez |
| PG native (Enterprise) | libpq | — | Enterprise | C seviyesi PG bağlantısı |
| Oracle native (Enterprise) | OCI — Oracle C Interface | — | Enterprise | C seviyesi Oracle bağlantısı |
| MSSQL native (Enterprise) | SQL Server Native Client | — | Enterprise | C seviyesi MSSQL bağlantısı |
| Konfigürasyon | YAML | — | C+E | PyYAML >= 6.0 |
| Bellek testi | Valgrind | — | Enterprise | C extension leak testi; CI/CD'de zorunlu |
| Lisans modülü | RSA şifreli | — | Enterprise | Binary wheel içinde; lisans doğrulama |
| Debug | debugpy | — | Geliştirme | VS Code remote debug; port 5678 |
| Loglama | Airflow DAG Logs (JSON) | — | C+E | ELK Stack / Splunk entegrasyonu için JSON format |

## Referans Ortam

| Bileşen | Değer |
|---|---|
| İşletim Sistemi | Linux (Ubuntu 24.04) |
| CPU | 8 vCPU |
| RAM | 32 GB |

| Bileşen | Değer |
|---|---|
| Docker Image | apache/airflow:3.1.6-python3.12 |
| PG Sürücüsü | psycopg3 3.x (binary COPY desteği için) |
| MSSQL Sürücüsü | pyodbc + msodbcsql18 (UnixODBC üzerinden) |
| Oracle Sürücüsü | python-oracledb (Thin mode — Oracle Client gerekmez) |

# 12 — Gereksinim ve Kapsam Sınır Tablosu

## Fonksiyonel Gereksinimler — Agent Referans Tablosu

| FR ID | Kapsam | Gereksinim | Öncelik | Agent Notu |
|---|---|---|---|---|
| FR01 | C | YAML tabanlı konfigürasyon ile ETL job oluşturma ve çalıştırma | P0 | Tüm config üretiminin temeli |
| FR02 | C | PostgreSQL, MSSQL, Oracle arasında çift yönlü veri aktarımı | P0 | Dialect sınıfı gerektirir |
| FR03 | C | Server-side cursor ile sabit bellek kullanımlı veri çekme | P0 | fetchmany(10000) zorunlu |
| FR04 | C | Kaynak metadata'dan otomatik DDL üretimi | P0 | Auto-DDL Generator |
| FR05 | E | Binary streaming: PostgreSQL COPY FROM STDIN | P0 | Enterprise Only |
| FR06 | C | Apache Airflow FFEngineOperator entegrasyonu | P0 | Her iki versiyonda ortak |
| FR07 | C | Partitioning desteği (5 mod) | P1 | auto/percentile/ hash_mod/explicit/distinct |
| FR08 | C | Binding parametreleri ile dinamik WHERE | P1 | source/target/literal/ airflow_var |
| FR09 | C | Backpressure yönetimi | P1 | Community: generator \| Enterprise: queue |
| FR10 | C | Auto-DAG Generator | P1 | YAML → Airflow DAG |
| FR11 | C | ETL Studio UI | P1 | Admin > ETL Studio menüsü |
| FR12 | E | Checkpoint & Resume (PK/Offset bazlı) | P0 | Enterprise Only — Queue Runtime |
| FR13 | E | Dead Letter Queue (DLQ) | P0 | Enterprise Only — Queue Runtime |
| FR14 | C | Dry-Run modu | P2 | DDL + bağlantı testi; veri aktarımı yok |
| FR15 | C | XCom metrik aktarımı (rows/duration/throughput) | P1 | FFEngineOperator.xcom_ push() |
| FR16 | E | Queue-Centric hedef yazma: kontrollü mikro-batch commit | P0 | Enterprise Only |
| FR17 | E | Multi-Lane Pipeline: paralel bağımsız akışlar | P1 | Enterprise Only |
| FR18 | E | Adaptif Batch Policy (row+byte+time+feedback) | P1 | Enterprise Only |
| FR19 | E | FFEnvelope: Ack/Nack + at-least-once + exactly-once (koşullu) | P1 | Enterprise Only — Guarantee Matrix |
| FR20 | E | Oracle AQ / TxEventQ source ve sink adapter | P2 | Enterprise — gelecek faz |

| FR ID | Kapsam | Gereksinim | Öncelik | Agent Notu |
|---|---|---|---|---|
| FR21 | E | PostgreSQL Queue Table (SKIP LOCKED) sink | P2 | Enterprise — gelecek faz |
| FR22 | E | COPY TO STDOUT (FORMAT BINARY) — PG binary extraction | P0 | Enterprise Only |
| FR23 | E | Asimetrik parametrik worker (reader_workers + writer_workers) | P0 | Enterprise Only |
| FR24 | C | 3 Fazlı DAG Pattern (plan → prepare → run) | P0 | Her iki versiyonda ortak |
| FR25 | C | column_mapping_mode (source + mapping_file) | P0 | Her iki versiyonda ortak |
| FR26 | C | TriggerDagRunOperator kademeli DAG tetikleme | P1 | Level1 → Level2 |
| FR27 | C | validate_config() çalışma öncesi doğrulama | P1 | Hatalı config'i erken yakalar |
| FR28 | C | source_type: script + load_method: script | P1 | Hedef DB'de SQL script çalıştırma |
| FR29 | C | sql_file parametresi ile harici .sql okuma | P1 | ETL Studio SQL ayrı dosyaya yazar |
| FR30 | C | Email Notification (on_failure_callback) | P1 | Türkçe HTML format |
| FR31 | C | ETL Studio: tablo keşfi, DAG güncelleme, tag, timeline | P1 | 50 tablo limiti + typeahead |
| FR32 | C | Transformer TABLE_RULES | P2 | strip/parse_dates/ to_numeric/ empty_to_null/ add_etl_date |

## Performans Hedefleri (NFR)

| NFR ID | Gereksinim | Community Hedef | Enterprise Hedef |
|---|---|---|---|
| NFR01 | RAM kullanımı (veri hacminden bağımsız) | < 500MB | < 256MB |
| NFR02 | Throughput (batch aktarım, queue overhead olmadan) | 100,000+ satır/sn | 500,000+ satır/sn |
| NFR03 | Throughput (tam queue semantiği) | N/A | 250,000–400,000 satır/sn |
| NFR04 | 10M+ satır aktarım süresi | < 3 dakika | < 1 dakika |
| NFR05 | Eş zamanlı job yürütme | Airflow Dynamic Task Mapping | Airflow Dynamic Task Mapping + Multi-Lane |
| NFR06 | Veri tutarlılık ve doğruluk | % 100 | % 100 |
| NFR07 | Yeni Dialect ekleme süresi | < 1 hafta | < 1 hafta |
| NFR08 | Binary streaming CPU tasarrufu | — | % 30–40 |

| NFR ID | Gereksinim | Community Hedef | Enterprise Hedef |
|---|---|---|---|
| NFR09 | Enterprise / Community hız oranı | 1x (referans) | 5x – 10x+ |

## Kapsam Dışı — Kesin Sınırlar

| Kapsam Dışı Konu | Sınıflandırma | Gerekçe |
|---|---|---|
| Change Data Capture (CDC / WAL / Redo Log) | Kalıcı kapsam dışı | ffengine High-Speed Batch odaklıdır; CDC farklı mimari paradigma |
| İş kuralı bazlı veri kalitesi (null, regex) | Kalıcı kapsam dışı | Akış anında tip doğruluğu dışında kapsam genişler |
| Veri maskeleme / anonimleştirme (KVKK/GDPR) | Kalıcı kapsam dışı | Ayrı ürün/süreç gerektirir |
| NoSQL veritabanları (MongoDB, Cassandra) | Kalıcı kapsam dışı | İlk sürümde RDBMS odaklı |
| Veri kataloğu / data governance | Kalıcı kapsam dışı | Ayrı ürün kapsamı |
| Kafka / RabbitMQ entegrasyonu | Ertelenmiş (gelecek faz) | Enterprise gelecek fazında değerlendirilecek |
| Snowflake / BigQuery connector | Ertelenmiş (2027+) | Enterprise C-connector olarak planlanıyor |
| MySQL Dialect | Ertelenmiş (2027+) | İlk sürümde PG/Oracle/MSSQL odaklı |
| On-premise → Cloud Hybrid Streaming | Ertelenmiş (2027+) | Enterprise gelecek faz |

**Agent İçin Son Kural:** 'Kalıcı kapsam dışı' konular için skill dosyası üretilmemelidir. 'Ertelenmiş' konular için 'gelecek faz' notu içeren bir placeholder skill üretilebilir ancak implementasyon kodu eklenmemelidir.